

A Taxonomy of Load Balancing in Data Center Networks

With Implications for QUIC Server-Side Connection Migration

Research Reference Document • June 2026

This document presents a comprehensive taxonomy of load balancing techniques used in modern data center networks. We classify approaches along six orthogonal dimensions: network layer, decision location, statefulness, algorithm, scope, and timing. For each category, we identify seminal papers from top systems conferences (SIGCOMM, NSDI, OSDI) and recent advances. We conclude by positioning QUIC server-side connection migration as a novel category in this taxonomy: **reactive, mid-connection, server-initiated load balancing** that requires no specialized hardware or client cooperation.

1. Introduction

Load balancing is fundamental to data center operations. As global data center traffic approaches 20.6 ZB/year (Cisco, 2025), distributing workloads efficiently across servers, paths, and regions has become critical for performance, availability, and cost. The rise of QUIC (RFC 9000) as the transport for HTTP/3 introduces both challenges and opportunities: QUIC's encrypted headers and connection migration break traditional L4 load balancing assumptions, but its **preferred_address** mechanism enables an entirely new form of server-initiated, mid-connection load balancing.

2. Taxonomy of Load Balancing

We classify load balancing along six orthogonal dimensions. A real-world deployment typically combines choices from multiple dimensions (e.g., Google uses L3 Anycast + L4 Maglev + L7 Envoy in a multi-tier architecture).

2.1 Dimension 1: By Network Layer

The OSI layer at which the load balancer inspects packets determines what information is available for routing decisions and how much overhead is incurred.

Layer	What It Inspects	Example Systems	Key Paper
L2 (Link)	MAC addresses, VLANs	Link Aggregation (LACP), bonding	IEEE 802.3ad
L3 (Network)	IP addresses only	ECMP, Anycast, BGP-based	RFC 2992 (ECMP)
L4 (Transport)	IP + port + protocol (TCP/UDP)	Maglev, Beamer, SilkRoad, IPVS, Katran	Maglev (NSDI '16)
L7 (Application)	HTTP headers, URLs, cookies, gRPC	NGINX, HAProxy, Envoy, AWS ALB	Envoy (2016)
L4.5 (QUIC-aware)	QUIC Connection IDs	QUIC-LB, QUIC server migration	QUIC-LB Draft (2025)

Note: QUIC blurs the L4/L7 boundary. Because QUIC encrypts transport headers, traditional L4 LBs cannot read ports after the handshake. QUIC-LB solves this by encoding routing info in Connection IDs. Our server migration approach goes further by moving the connection itself.

2.2 Dimension 2: By Decision Location

Where the load balancing decision is made affects latency, scalability, and failure modes.

Location	Description	Examples	Trade-off
DNS-based	Resolve domain name to different server IPs	Route53, Cloudflare DNS, GSLB	Simple but coarse-grained; TTL caching delays reaction

Location	Description	Examples	Trade-off
Anycast	Same IP advertised from multiple locations; BGP routing picks closest	CDNs, Google VIPs	No per-connection state; limited to geographic proximity
Network / Fabric	Switches and routers distribute across paths	ECMP, CONGA, PLB	Low latency; limited to path selection, not server selection
Dedicated LB (HW)	Dedicated appliance in front of backend pool	F5 BIG-IP, Citrix ADC	High throughput; expensive, single point of failure
Dedicated LB (SW)	Software process in front of backend pool	Maglev, HAProxy, NGINX, Envoy	Flexible; adds latency, needs scaling itself
In-Switch LB	LB logic programmed into switching ASICs (P4)	SilkRoad, Cheetah (HW mode)	Line-rate; limited memory for state
Server-Side (Self-Balancing)	Server itself decides to shed load by migrating connections	QUIC preferred_address migration	No LB bottleneck; requires protocol support

2.3 Dimension 3: By State Management

Type	Description	Examples	Resilience
Stateless	No per-connection memory in the LB; every packet routed independently	ECMP, Beamer, Cheetah (stateless)	Survives LB failures; may break conn affinity
Stateful	LB tracks each connection in a table	Maglev, SilkRoad, iptables CONNTRACK	Guarantees PCC; state = vulnerability (SYN floods)
Soft-State	State exists but can be reconstructed from hashing	Consistent hashing (Maglev ring)	Graceful degradation; small % of conn disrupted
State-Transferred	Connection state explicitly moved between servers	QUIC server migration (445 bytes)	State lives on servers, not LB; LB can be stateless

2.4 Dimension 4: By Algorithm / Policy

Algorithm	How It Selects	Best For	Complexity
Round-Robin	Next server in cyclic order	Equal-capacity servers, stateless	O(1)
Weighted Round-Robin	Proportional to assigned weights	Heterogeneous server capacity	O(1)
Least Connections	Server with fewest active connections	Variable request durations	O(log n)

Algorithm	How It Selects	Best For	Complexity
Least Response Time	Server responding fastest	Latency-sensitive applications	$O(n)$ probing
Random	Uniformly random selection	Simple, zero-state	$O(1)$
Power of 2 Choices	Pick 2 random servers, choose less loaded	Near-optimal with minimal overhead	$O(1)$
Consistent Hashing	hash(key) maps to server ring; minimal disruption on changes	Caches, session affinity	$O(\log n)$
Congestion-Aware	Real-time load/congestion signals steer traffic	Fabric-level path selection	$O(1)$ per switch
Resource-Aware	CPU / memory / GPU utilization metrics	Heterogeneous workloads (ML, etc.)	$O(n)$ monitoring

2.5 Dimension 5: By Scope

Scope	Description	Examples	Typical Mechanism
Global (GSLB)	Across data centers and geographic regions	DNS-based GSLB, Anycast	DNS, BGP
Data Center	Within a single data center	Maglev, Beamer, fabric ECMP	L4 LB, ECMP
Intra-Rack	Within a single rack	Top-of-Rack switch ECMP	Switch hashing
Per-Service (Mesh)	Between microservice instances	Envoy, Istio, Linkerd	Sidecar proxy, L7
Per-Connection	Individual connection migrated to a different server	QUIC server migration	preferred_address + state transfer

2.6 Dimension 6: By Timing

This is the most important dimension for understanding why QUIC server migration is novel. Almost all existing load balancing happens **before** or **at** connection establishment. Once a connection is pinned to a backend, it stays there until it ends.

Timing	When Decision Occurs	Examples	Can Change Server?
Pre-Connection	Before any packets are sent (DNS resolution time)	DNS-based GSLB, Anycast routing	N/A (no connection yet)
At Connection	At SYN / QUIC Initial packet time	All L4 LBs (Maglev, Beamer, SilkRoad, Katran)	No (pinned after decision)
Flowlet-Level	During connection, at flowlet boundaries	CONGA, PLB	Changes <i>path</i> , not <i>server</i>
Mid-Connection (Reactive)	After connection is established, server-initiated	QUIC preferred_address migration	Yes -- transparently moves to different server

Key insight: QUIC server-side migration is the only mechanism in this taxonomy that can move an established, encrypted connection to a completely different physical server without client awareness and without any specialized load balancing infrastructure.

3. Key Systems and Papers

Below we summarize the most influential load balancing systems from top venues, organized by their approach.

3.1 Layer-4 Software Load Balancers

Maglev (Google) -- NSDI 2016

Google's production L4 load balancer. Uses consistent hashing with a novel "Maglev hash" that provides minimal disruption and uniform load distribution. Deployed on commodity Linux servers behind ECMP routers. Each Maglev machine can independently route any packet. Per-connection state via connection tracking table provides affinity.

- Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer," NSDI 2016. <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud>

Beamer -- NSDI 2018

Eliminates per-connection state from the LB mux entirely. When a backend pool changes, Beamer "daisy-chains" old backends to forward residual packets to new backends. Achieves 33M pps (2x Maglev) and is immune to SYN flood state exhaustion attacks.

- Olteanu et al., "Stateless Datacenter Load-balancing with Beamer," NSDI 2018. <https://www.usenix.org/conference/nsdi18/presentation/olteanu>

Katran (Meta) -- Open Source, 2018

Meta's production L4 LB using XDP/eBPF for kernel-bypass packet processing. Handles QUIC connection IDs for routing. Open-source. Runs on commodity hardware at line rate.

- Facebook Engineering, "Open-sourcing Katran." <https://engineering.fb.com/2018/05/22/open-source/open-sourcing-katran-a-scalable-network-load-balancer/>

3.2 In-Switch / Hardware Load Balancers

SilkRoad -- SIGCOMM 2017

Implements stateful L4 LB in a switching ASIC using 400 lines of P4. Supports 10M concurrent connections at line rate. Replaces hundreds of software LB servers with a single switch, reducing cost by 100x.

- Miao et al., "SilkRoad: Making Stateful Layer-4 Load Balancing Fast and Cheap Using Switching ASICs," SIGCOMM 2017. <https://dl.acm.org/doi/10.1145/3098822.3098824>

Cheetah -- NSDI 2020

Encodes routing info in a cookie embedded in transport headers (stateless mode) or uses hash tables (stateful mode). Implemented on both software and Tofino ASIC. Guarantees per-connection consistency with 2-3x better flow completion times.

- Barbette et al., "A High-Speed Load-Balancer Design with Guaranteed Per-Connection-Consistency," NSDI 2020. <https://www.usenix.org/system/files/nsdi20-paper-barbette.pdf>

3.3 Fabric-Level / Congestion-Aware

CONGA -- SIGCOMM 2014

Distributed congestion-aware LB implemented in custom ASICs (Cisco). Splits flows into flowlets and routes them based on real-time congestion feedback from remote leaf switches. 5x better FCT than ECMP under asymmetry.

- Alizadeh et al., "CONGA: Distributed Congestion-Aware Load Balancing for Datacenters," SIGCOMM 2014. <https://dl.acm.org/doi/10.1145/2740070.2626316>

PLB (Google) -- SIGCOMM 2022

Google's production path-level LB. Much simpler than CONGA: uses ECN congestion signals to reroute flowlets away from congested paths. Deployed at scale across Google's data centers. Demonstrates that simple signals suffice for production environments.

- PLB: Congestion Signals are Enough, SIGCOMM 2022. <https://dl.acm.org/doi/10.1145/3544216.3544226>

3.4 QUIC-Specific Load Balancing

QUIC-LB -- IETF Draft (Active, 2025)

The IETF's standardized approach to QUIC load balancing. Servers encode their server ID into QUIC Connection IDs using a shared key with the LB. The LB decodes the CID to route packets to the correct backend -- even after client address migration. Does NOT support moving a connection to a different server; only routing to the original server.

- Duke et al., "QUIC-LB: Generating Routable QUIC Connection IDs," IETF Draft v21. <https://datatracker.ietf.org/doc/draft-ietf-quic-load-balancers/>

QASM: QUIC-Aware Stateful Middleboxes -- 2025

A framework for building middleboxes (including LBs) that understand QUIC's connection migration. Addresses the challenge that a single connection must remain mapped to a single server across CID changes.

- "QASM: A Novel Framework for QUIC-Aware Stateful Middleboxes," 2025. <https://arxiv.org/pdf/2602.03354>

Analysis of QUIC Connection Migration in the Wild -- 2025

Empirical study of QUIC connection migration support across the Internet. Reveals that despite rapid QUIC deployment, many popular destinations do not support connection migration.

- "An Analysis of QUIC Connection Migration in the Wild," ACM SIGCOMM CCR 2025. <https://dl.acm.org/doi/10.1145/3727063.3727066>

Server-Side QUIC Migration for Edge Microservices -- 2022

Proposes extending QUIC for server-side migration when containers are migrated between edge nodes. Three strategies (orchestrator-assisted, peer-to-peer, DNS-based). Evaluated against TCP+DNS and MPTCP baselines. **Most directly related to our work**, but limited to simulation -- our implementation demonstrates real cross-machine migration with Firefox.

- Puliafito et al., "Server-side QUIC connection migration to support microservice deployment at the edge," Pervasive and Mobile Computing, 2022. <https://www.sciencedirect.com/science/article/abs/pii/S157411922200030X>

QUIC-Exfil: Security Implications -- ASIA CCS 2025

Demonstrates that QUIC's preferred_address mechanism can be exploited for data exfiltration. Firewalls cannot distinguish malicious migrations from legitimate ones. Five ML classifiers failed to detect the attack (0-47% F1-score). Our implementation provides the first cross-machine proof of this attack's feasibility.

- Grubl et al., "QUIC-Exfil: Exploiting QUIC's Server Preferred Address Feature," ASIA CCS 2025. <https://arxiv.org/pdf/2505.05292>

4. Positioning QUIC Server Migration in the Taxonomy

The following table compares our QUIC server-side migration approach against the major load balancing systems across the six taxonomy dimensions.

System	Layer	Location	State	Timing	Can Move Server?
Maglev	L4	SW LB	Stateful	At connection	No
Beamer	L4	SW LB	Stateless	At connection	No
SilkRoad	L4	In-switch	Stateful	At connection	No
Cheetah	L4	In-switch/SW	Both	At connection	No
CONGA	L3	Fabric	Stateless	Flowlet-level	No (path only)
PLB	L3	Fabric	Stateless	Flowlet-level	No (path only)
QUIC-LB	L4.5	SW LB	Stateless	At connection	No (routes to original)
Envoy/Istio	L7	Sidecar	Stateful	At request	Per-request only
QUIC Migration	L4.5	Server	State-transfer red	Mid-connection	Yes

4.1 Data Center Use Cases Enabled by Mid-Connection Migration

- **Zero-Downtime Server Drain:** A server scheduled for maintenance advertises preferred_address for new handshakes, migrating active connections to healthy backends. No connection drops, no client-visible errors.
- **Reactive Load Shedding:** Unlike traditional LBs that pin connections at establishment, an overloaded server can shed load mid-connection. A load controller writes target addresses to Redis; servers pick up migration targets dynamically.
- **Geographic Re-routing:** Anycast DNS routes the client to the nearest PoP for a fast handshake. The server then migrates the connection to a backend with the client's data (potentially in a different region).
- **LB Bottleneck Elimination:** The LB only handles initial handshakes. Post-migration, clients communicate directly with backends. This is the "load balancer bypass" use case described in RFC 9000 Section 9.6.
- **Edge Container Migration:** When a microservice container is migrated between edge nodes (for user mobility), QUIC migration keeps the connection alive without TCP re-establishment (Puliafito et al., 2022).

5. Open Research Questions

- **Migration Scheduling Policy:** Who decides when and where to migrate? Centralized controller (Kubernetes-style), or autonomous server decision? What metrics trigger migration (CPU, queue depth, latency percentile)?
- **Multi-Hop Migration:** RFC 9000 allows only one preferred_address per handshake. For continuous rebalancing, NEW_CONNECTION_ID + subsequent path migration would be needed. No implementation

supports this yet.

- **State Transfer Security:** The 445-byte migration state contains TLS secrets. Securing this transfer (mTLS, encrypted Redis, etc.) is critical. Our analysis shows Redis KV is best for security/scalability, HTTP Pull for secrets-never-leave-memory guarantees.
- **Interaction with QUIC-LB:** Can QUIC-LB CID encoding coexist with preferred_address migration? The migrated server would need to generate CIDs compatible with the LB's decoding scheme.
- **Scalability Limits:** How many simultaneous migrations can a data center sustain? State transfer is small (445 bytes), but PATH_CHALLENGE/RESPONSE adds 1 RTT per migration.
- **Client Behavior Variance:** Different QUIC implementations handle preferred_address differently. Firefox sends multiple PATH_CHALLENGEs; other clients may not. Standardization of client migration behavior is incomplete.

6. Complete Reference List

Papers organized by venue and relevance to QUIC server-side migration.

Top Conference Papers

- [1] Alizadeh et al., "CONGA: Distributed Congestion-Aware Load Balancing for Datacenters," **SIGCOMM 2014**. <https://dl.acm.org/doi/10.1145/2740070.2626316>
- [2] Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer," **NSDI 2016**. <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud>
- [3] Miao et al., "SilkRoad: Making Stateful Layer-4 Load Balancing Fast and Cheap Using Switching ASICs," **SIGCOMM 2017**. <https://dl.acm.org/doi/10.1145/3098822.3098824>
- [4] Olteanu et al., "Stateless Datacenter Load-balancing with Beamer," **NSDI 2018**. <https://www.usenix.org/conference/nsdi18/presentation/olteanu>
- [5] Araujo et al., "Balancing on the Edge: Transport Affinity without Network State," **NSDI 2018**. <https://www.usenix.org/system/files/conference/nsdi18/nsdi18-araujo.pdf>
- [6] Barbette et al., "A High-Speed Load-Balancer Design with Guaranteed Per-Connection-Consistency (Cheetah)," **NSDI 2020**. <https://www.usenix.org/system/files/nsdi20-paper-barbette.pdf>
- [7] "PLB: Congestion Signals are Enough," **SIGCOMM 2022**. <https://dl.acm.org/doi/10.1145/3544216.3544226>
- [8] Grubl et al., "QUIC-Exfil: Exploiting QUIC's Server Preferred Address Feature," **ASIA CCS 2025**. <https://arxiv.org/pdf/2505.05292>

IETF Standards and Drafts

- [9] RFC 9000, "QUIC: A UDP-Based Multiplexed and Secure Transport," IETF 2021. <https://www.rfc-editor.org/rfc/rfc9000.html>
- [10] RFC 9312, "Manageability of the QUIC Transport Protocol," IETF 2022. <https://www.rfc-editor.org/rfc/rfc9312.html>
- [11] Duke et al., "QUIC-LB: Generating Routable QUIC Connection IDs," IETF Draft v21, 2025. <https://datatracker.ietf.org/doc/draft-ietf-quic-load-balancers/>
- [12] Tan et al., "QUIC Connection Migration," IETF Draft, 2024. <https://datatracker.ietf.org/doc/html/draft-tan-quic-connection-migration-00>

Journal Papers and Other

- [13] Puliafito et al., "Server-side QUIC connection migration to support microservice deployment at the edge," **Pervasive and Mobile Computing**, 2022. <https://www.sciencedirect.com/science/article/abs/pii/S157411922200030X>
- [14] "An Analysis of QUIC Connection Migration in the Wild," **ACM SIGCOMM CCR**, 2025. <https://dl.acm.org/doi/10.1145/3727063.3727066>
- [15] "QASM: A Novel Framework for QUIC-Aware Stateful Middleboxes," 2025. <https://arxiv.org/pdf/2602.03354>
- [16] "Enhancing QUIC Performance in Heterogeneous Networks: A Proactive Connection Migration Approach," **Int. J. Network Management**, 2025. <https://onlinelibrary.wiley.com/doi/10.1002/nem.70022>
- [17] Liu et al., "Traffic load balancing in data center networks: A comprehensive survey," **Computer Science Review**, 2025. <https://www.sciencedirect.com/science/article/abs/pii/S1574013725000255>
- [18] Facebook Engineering, "Open-sourcing Katran, a scalable network load balancer," 2018. <https://engineering.fb.com/2018/05/22/open-source/open-sourcing-katran-a-scalable-network-load-balancer/>